# Machine Learning

Compiled Revision Notes

# Table of Contents

**Week 1: Introduction and Regression** **4**
Introduction to Machine Learning . . . 4
Linear Regression . . . 4

**Week 2: Generalisation and Model Evaluation** **6**
Generalisation . . . 6
Testing and Validation Procedures . . . 6
Model Selection and Hyperparameters . . . 7

**Week 3: Classification & Logistic Regression** **8**
Introduction to Classification . . . 8
Logistic Regression & The Logit Function . . . 8
Training & Loss Functions . . . 11
Metrics & Evaluation . . . 11

**Week 4: k-Nearest Neighbours, Feature Engineering & Tree Ensembles** **13**
k-Nearest Neighbours (kNN) . . . 13
Lazy vs. Eager Learning . . . 13
Feature Engineering & Preprocessing . . . 14
The Curse of Dimensionality . . . 14
Decision Trees & Ensembles . . . 15
AdaBoost (Adaptive Boosting) . . . 17
AdaBoost vs. Random Forest . . . 17

**Week 5: Gradient Descent** **17**
Training and Loss Functions . . . 17
The Gradient Descent Algorithm . . . 18
Gradient Descent Variations . . . 19
Epochs, Batches, and Learning Rates . . . 20

**Week 6: Regularisation and Support Vector Classifiers** **21**
Regularisation . . . 21
Support Vector Classifiers . . . 22

**Week 7: Support Vector Classifiers and Machines** **23**
Margins and Decision Boundaries . . . 23
Soft Margin Classifiers and Slack Variables . . . 24
Hinge Loss . . . 25
Support Vector Machines and Higher Dimensions . . . 25
The Kernel Trick . . . 25
Types of Kernels . . . 26
Implementation Tips . . . 26

**Week 8: Neural Networks and Deep Learning Frameworks** **26**

Artificial Neural Networks . . . 26
The Single Perceptron . . . 27
Activation Functions . . . 27
Network Topology . . . 28
Deep Learning Frameworks: TensorFlow and Keras . . . 28
Building and Training Models in Keras . . . 29

**Week 9: Convolutional Neural Networks** **29**
Background and Motivation . . . 29
CNN Concepts and Layers . . . 30
CNN Training and Overfitting Prevention . . . 31
Famous Architectures . . . 31

**Week 10: Neural Networks with Multiple Outputs & Transfer Learning** **32**
Neural Networks with Multiple Outputs . . . 32
Transfer Learning . . . 34

**Week 11: Backpropagation and Neural Network Training Parameters** **35**
Backpropagation . . . 35
Neural Network Training Parameters . . . 35

# Week 1: Introduction and Regression

## Introduction to Machine Learning

### What is Machine Learning?

- **Definition:** A field of study giving computers the ability to learn without being explicitly programmed. It is about changing behavior to improve performance in the future.
- **Core Concept:** Instead of hand-crafting complex rules (lots of `if` statements), we use data to fit parameters of a model.
  - **The Equation:** We want to find the function $f$ in the equation $y = f(x)$, where we figure out the right-hand side by gathering data and fitting parameters.

### Types of Learning

- **Supervised Learning:**
  - **Definition:** Learning a function mapping inputs to outputs based on training examples where we already know the correct result (labeled data).
  - **Classification:** The output is a category or discrete label (e.g., Face/No Face, Digit 0–9).
  - **Regression:** The output is a continuous value (e.g., Stock prices, Turkey cooking time).
- **Unsupervised Learning:**
  - Techniques where there is no "right" answer known; the algorithm tries to find structure or patterns in the data.
  - Examples: Clustering (grouping data), Dimensionality Reduction.
- **Reinforcement Learning:** Learning through interaction (e.g., AlphaGo playing games against itself).

### The ML Pipeline

To perform Machine Learning, you need 4 components:

1. **The Task:** Define or limit what you are trying to do.
2. **The Experience:** The data (more data = more experience).
3. **The Performance Measure:** A way to measure success.
4. **The Learning Algorithm:** The recipe by which we will improve our performance.

---

## Linear Regression

### Variables

- **Independent Variable ($x$):** The input. It changes independently (e.g., Time).
- **Dependent Variable ($y$):** The output. It depends on the input (e.g., House Value).
- **Visualizing:** Plot independent variables on the horizontal axis ($x$) and dependent on the vertical axis ($y$).

### Linear Relationships

- **Definition:** A linear relationship means a change in $x$ always produces the same proportionate change in $y$.

- **School Math Formula:** $y = mx + c$, where $m$ is the slope and $c$ is the y-intercept.
- **Machine Learning Notation:**
  We scale this up for larger models using **Weights ($w$)**. Weights are the parameters or coefficients that the model learns during training.

$$y = w_0 + w_1 x$$

  - $w_0$ = bias or intercept (previously $c$).
  - $w_1$ = weight or slope (previously $m$).

**The Turkey Example**

- **Scenario:** Predicting cooking time ($t$) based on weight ($w$).
- **Linear Model:** $t = mw + c$ (Rule of thumb: 20 mins per pound + 20 mins).
  - *Critique:* Implies you cook a 0 lb turkey for 20 mins (the y-intercept).
- **Power Model (Pief Panofsky):**

$$t = \frac{w^{2/3}}{1.5}$$

  - *Comparison:* Linear works well for small weights (1–6 lbs) but diverges significantly at higher weights.
- **Lesson:** Data points might follow a curve. If data is scattered on a curve, a simple linear model is an **inappropriate fit** as it assumes values keep increasing indefinitely.

**Training the Model**

- **Goal:** Find the best parameters $(w_0, w_1)$ that minimize the error for our training sample.
- **Prediction vs. Observation:**
  - $y$ **(Observation):** The actual observed data point.
  - $\hat{y}$ **(Prediction):** The value predicted by the function.
  - **Residual:** The difference between the prediction ($\hat{y}$) and the observation ($y$).
- **Cost Function (Mean Squared Error, MSE):**

$$L = \frac{1}{2m} \sum_{i=0}^{m} (\hat{y}_i - y_i)^2$$

  - $m$: number of samples.
  - Note: The $\frac{1}{2m}$ term is for mathematical convenience to make derivatives simpler.
- **Gradient Descent:**
  - The algorithm typically used to solve for the parameters ($w$).
  - It performs multiple iterations to find the parameters that give the smallest loss ($L$).

#### Multiple Linear Regression

Real-world problems have more than one input feature.

- **Polynomial Regression:** $y = w_0 + w_1 x + w_2 x^2 + ...$ (fits curves).
- **Multiple Linear Regression:** Input $x$ is a vector of features $(x_1, x_2, ...)$.

$$y = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3 + ...$$

---

## Week 2: Generalisation and Model Evaluation

### Generalisation

#### The Goal of Machine Learning

- **Definition:** Generalisation is the model's ability to give sensible outputs to sets of input that it has never seen before.
- It is not enough to perform well on training data; the model must perform well on **previously unseen data**.
- **Measurement:** We often use root-mean-squared (RMS) error to measure performance, but calculating this on training data is misleading because the model might be overfit.

#### Underfitting vs. Overfitting

- **Underfitting (High Bias):**
  - Occurs when the model is too simple to capture the underlying trend of the data.
  - **Symptoms:** Poor performance on both the training data and the test data. The model has not learned enough.
- **Overfitting (High Variance):**
  - Occurs when the model fits the training data too well, capturing noise rather than the signal.
  - **Symptoms:** Very low training error, but high test error. The model is unreliable on new data.
  - **Confidence:** Overfit models often predict incorrect results with very high confidence.

#### Bias and Variance Trade-off

- **Bias:** Making assumptions about the underlying model. High bias leads to underfitting.
- **Variance:** Sensitivity to training data. High variance leads to overfitting.
- **The Goal:** Find the sweet spot between high bias and high variance to ensure good generalisation.

---

### Testing and Validation Procedures

#### Train and Test Split

- To test a model, we split available data into two distinct sets:

1. **Training Set:** Used to fit the model and learn parameters such as weights.
2. **Test Set:** Used only to evaluate performance and acts as unseen data.

- **Typical Splits:** 80/20 or 75/25 (Train/Test).
- **Crucial Rule:** The test set must be guarded carefully and used sparingly/carefully. If you use it to make decisions such as choosing model complexity, it is no longer unseen and the evaluation becomes invalid.

### Data Leakage

- **Definition:** Leakage occurs when information from the test or validation set unintentionally influences the training process. This leads to overly optimistic performance estimates that fail in the real world.
- **The Cause:** Often caused by non-independent samples. If correlated data appears in both training and testing sets, the model effectively memorizes the answer via hidden context.
- **Example: Bird Call Recordings**
  - *Scenario:* Recordings from Jan 1st (Recording A, Recording B), Jan 2nd, etc.
  - *Bad Split:* Randomly assigning files. Recording A goes to Train, Recording B goes to Test. The model learns background noise rather than bird species.
  - *Correct Approach:* **Group-based splitting.** Split by day or week. If Jan 1st is in Test, all recordings from Jan 1st must be in Test.
- **Key Takeaway:** Ensure split groups are meaningful, such as blocks of time, to test true generalisation.

---

## Model Selection and Hyperparameters

### Hyperparameters

- **Definition:** Parameters that are chosen by the engineer, not learned by the algorithm.
- **Examples:**
  - Degree of polynomial (linear vs quadratic vs cubic).
  - Learning rate ($a$).
  - Number of layers in a neural network.

### The Validation Set

- We cannot use the **test set** to choose hyperparameters.
- **Solution:** Split training data further into training and validation sets (often 70/30).
- **Procedure:**
  1. Train different models on the training portion.
  2. Check error on the validation portion.
  3. Pick the hyperparameter with the lowest validation error.
  4. **Final Step:** Evaluate the chosen model on the test set, which has remained hidden.

### Cross-Validation (K-Fold)

- **Problem:** A single validation split might be biased.
- **Solution (K-Fold):**
  1. Split data into $k$ equal folds (e.g., $k = 5$ or $k = 10$).

2. Train on $k-1$ folds and validate on the remaining fold.
3. Repeat $k$ times so each fold is used as validation once.
4. Average the error across all runs to obtain a robust metric.

- **Benefit:** Reduces the risk that a specific random split skews results.

#### Polynomial Regression

- Complex curved data can be modeled by adding powers of $x$ (e.g., $x^2, x^3$).
- **Degree ($M$):** The hyperparameter controlling complexity.
  - Low $M$ (e.g., 1): High bias, underfitting.
  - High $M$ (e.g., 9): High variance, overfitting.
- Cross-validation is used to find the optimal $M$.

## Week 3: Classification & Logistic Regression

### Introduction to Classification

#### What is Classification?

- **Definition:** Unlike regression which predicts continuous values, classification predicts which **category** or group a sample belongs to.
- **Outputs:** The output is restricted to a limited set of possible classes (e.g., 0 or 1).
- **Examples:** Spam detection (Spam/Not Spam), Fraud detection, Medical diagnosis (Malignant/Benign).

#### Binary Classification

- **Definition:** A problem with only two classes:
  - **Negative Class (0):** The absence of the event (e.g., Benign).
  - **Positive Class (1):** The presence of the event (e.g., Malignant).
- **Categorical Data:** We represent categorical outcomes numerically, typically as 0 and 1.

**Lab Note: Encoding Categorical Features** When converting text labels to numbers (Encoding), be careful: - **Label Encoding:** Assigning numbers (0, 1, 2) can mislead the model into thinking there is an order or hierarchy (e.g., $2 > 1$), which might not exist. - **One-Hot Encoding:** A safer approach for non-ordinal data. It splits one feature into multiple binary features (e.g., `Color_Red`, `Color_Blue`), preventing the model from assuming false relationships.

---

### Logistic Regression & The Logit Function

Logistic Regression is a **classification algorithm**, named "regression" only because of its mathematical roots. Despite its name, we are not trying to predict a continuous value. The response variable $y$ is either 0 or 1.

We do predict a continuous value between 0 and 1, but we interpret it as a probability ($p$) and then apply a threshold to obtain a discrete class label.

---

### The Core Problem with Linear Regression

With standard linear regression:

$$\hat{y} = w_0 + w_1 x_1$$

- Predictions range from $-\infty$ to $+\infty$.
- For binary classification, $\hat{y}$ can easily be $< 0$ or $> 1$.
- We want an estimate of probability $p$ strictly between 0 and 1.

Linear regression therefore does not naturally constrain outputs to the valid probability range.

---

### The Sigmoid Function

The **sigmoid function**, also called the **logistic function**, takes any real-valued number and maps it into the range $(0, 1)$.

$$y = f(x) = \frac{1}{1 + e^{-x}}$$

- As $x \to +\infty$, $y \to 1$.
- As $x \to -\infty$, $y \to 0$.
- The function maps values from $(-\infty, +\infty)$ to $(0, 1)$.

Since our linear model output

$$\hat{y} = w_0 + w_1 x_1 + \cdots + w_n x_n$$

also ranges from $-\infty$ to $+\infty$, we apply the sigmoid to map it into a probability:

$$p = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + \cdots + w_n x_n)}}$$

This is the **logistic regression function**. The weights $w_0, \ldots, w_n$ are often written as $\beta_0, \ldots, \beta_n$ in the literature.

---

### Odds and the Logit Function

The term

$$\frac{p}{1 - p}$$

is known as the **odds**.

Examples: - If $p = 0.5$: $\frac{0.5}{0.5} = 1 : 1$ - If $p = 0.8$: $\frac{0.8}{0.2} = 4 : 1$ - If $p = 0.05$: $\frac{0.05}{0.95} \approx 0.053 : 1$

The odds function is asymmetrical: - If $0 < p < 0.5$, odds lie between 0 and 1. - If $0.5 < p < 1$, odds lie between 1 and $\infty$.

To address this asymmetry, we take the natural logarithm of the odds:

$$\ln\left(\frac{p}{1-p}\right)$$

This is called the **logit function**:

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$$

The logit function forms the basis of logistic regression.

The linear model in logistic regression is therefore written as:

$$\ln\left(\frac{p}{1-p}\right) = w_0 + w_1x_1 + \cdots + w_nx_n$$

---

**Logit vs. Logistic Function**

The logit and logistic functions are inverses of each other.

| Feature | **Logistic (Sigmoid) Function** | **Logit Function** |
|---|---|---|
| **Formula** | $f(x) = \frac{1}{1+e^{-x}}$ | $\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$ |
| **Purpose** | Maps log-odds ($x$) to probability ($p$) | Maps probability ($p$) to log-odds ($x$) |
| **Input Range** | $-\infty < x < \infty$ | $0 < p < 1$ |
| **Output Range** | $0 < p < 1$ | $-\infty < x < \infty$ |
| **Use in Logistic Regression** | Converts linear model output to probability | Converts probability to a linear form |

**Decision Boundaries**

- **Thresholding:** The model outputs a probability (e.g., 0.7). We must choose a threshold (e.g., 0.5) to classify it as Class 1 or Class 0.
- **Trade-off:** This threshold is technically a hyperparameter, often chosen based on the problem (e.g., lowering the threshold for cancer diagnosis to minimize False Negatives).

**Is Threshold a Hyperparameter? - Short Answer:** It is complicated. Technically yes, but it is often determined by **human decision** and domain constraints rather than just data. - **Example:** In cancer diagnosis, you might lower the threshold (e.g., to 0.1) to catch more cases. You would rather tolerate False Positives (scaring a healthy patient) than False Negatives (missing cancer).

---

## Training & Loss Functions

### Why MSE Fails for Classification

- **Non-Convexity:** If you use Mean Squared Error (MSE) with the Sigmoid function, the resulting cost function is **non-convex**.
- **Result:** The graph looks "wavy" with many local minima, making it impossible for Gradient Descent to reliably find the global minimum (the best weights).

### Log Loss (Binary Cross-Entropy)

- We use **Log Loss** instead. It is **convex**, ensuring a single global minimum that makes optimization efficient.
- **The Logic:**
  - If correct answer is 1: We want predicted $p$ to be close to 1. If $p$ is low, error is massive ($-\log(p)$).
  - If correct answer is 0: We want predicted $p$ to be close to 0. If $p$ is high, error is massive ($-\log(1-p)$).
- **Formula:**

$$L(w) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i)]$$

---

## Metrics & Evaluation

### L1 vs. L2 Norms

These are ways to measure the "magnitude" or distance of a vector, often used in calculating errors.

- **L1 Norm (Manhattan):** The sum of absolute values.

$$||x||_1 = \sum |x_i|$$

- **L2 Norm (Euclidean):** The square root of the sum of squared values (straight line distance).

$$||x||_2 = \sqrt{\sum x_i^2}$$

### Regression Metrics

When evaluating regression models (predicting continuous values), we use several key metrics:

- **MSE (Mean Squared Error):**
  - **Formula:**

$$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

  - **Use:** Standard for **Training** loss because it is differentiable (convex).

  - **Pros/Cons:** Highly sensitive to outliers because it squares the error (large errors become huge).
- **RMSE (Root Mean Squared Error):**
  - **Formula:**

$$\sqrt{MSE}$$

  - **Use: Model Evaluation**.
  - **Pros:** It returns the error to the same units as the target variable (e.g., dollars, minutes), making it easier to interpret.
- **MAE (Mean Absolute Error):**
  - **Formula:**

$$\frac{1}{n}\sum |y_i - \hat{y}_i|$$

  - **Use: Model Evaluation**.
  - **Pros/Cons:** Less sensitive to outliers than MSE. However, it is **not differentiable** at 0 (has a sharp "kink"), so it cannot be used as a loss function for training algorithms like Gradient Descent.
- $R^2$ **(Coefficient of Determination):**
  - **Formula:**

$$1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}$$

  - **Interpretation:** Represents the proportion of variance in the dependent variable explained by the model (Unitless, max value is 1).

**Classification Metrics**

- **Accuracy:** Fraction of correct predictions. Good for balanced classes, misleading for imbalanced ones.
- **Confusion Matrix:** A table comparing Actual vs. Predicted values (TP, TN, FP, FN).
- **Precision:** $\frac{TP}{TP+FP}$ (Accuracy of positive predictions).
- **Recall:** $\frac{TP}{TP+FN}$ (Ability to find all positive instances).
- **F1-Score:** Harmonic mean of Precision and Recall. Best for imbalanced datasets.

**Trade off between Precision and Recall (PR Curve and PR AUC)** Precision and Recall usually trade off. Increasing the threshold increases Precision but lowers Recall (and vice versa). The PR Curve and the Area under the curve (AUC), can also be a good single number estimator of model performance.

**Note on Optimization** We typically **train** using one metric (like Log-Loss for classification or MSE for regression because they are differentiable/convex) but **evaluate** using others (like Accuracy, F1-Score, or MAE) that are more interpretable for humans.

# Week 4: k-Nearest Neighbours, Feature Engineering & Tree Ensembles

## k-Nearest Neighbours (kNN)

### The Algorithm

- **Definition:** kNN is a simple, two-part algorithm used for either classification or regression.
- **Procedure:**
  1. Given a target instance ($x_j$), calculate the distance to all recorded training cases ($x_i$).
  2. Retrieve the $k$ most similar (nearest) recorded cases.
  3. **For Classification:** Take the maximum of $k$ votes (the most common category among the neighbors).
  4. **For Regression:** Calculate the "average" across these $k$ cases.

### Distance Metrics

- We typically use **Euclidean distance** to calculate the "nearest" neighbor.
- **Formula:** $d(t, s) = \sqrt{(t_1 - s_1)^2 + ... + (t_p - s_p)^2}$.
- **The Scale Problem:** Scale matters significantly. If one predictor/feature has a much larger value range than another, it will disproportionately influence the distance measurement.

### Choosing $k$

- $k = 1$ **(1-Nearest Neighbour):** Looking at only the single closest point tends to overfit the training data and captures noise.
- **Higher** $k$ **(e.g.,** $k = 5$**):** Taking a vote among multiple neighbors smooths out the decision boundary and reduces overfitting.
- **The Trade-off:** If $k$ is too small, the model overfits. If $k$ is too large, the model underfits reality.
- **How to choose:** We treat $k$ as a hyperparameter and select the best value using cross-validation (e.g., 10-fold cross-validation).

## Lazy vs. Eager Learning

- **Lazy Learning (e.g., kNN):** Defers computation until a prediction is needed.
  - *Pros:* Very quick to train (it just stores the data), less memory usage during training.
  - *Cons:* Prediction is slow, relies heavily on training data during prediction, uses more memory during prediction.
- **Eager Learning (e.g., SVM, Neural Networks, Decision Trees):** Precomputes a model during the training phase.
  - *Pros:* Faster predictions, less memory usage during prediction, less dependent on original data once trained.
  - *Cons:* Slower and more memory-intensive during the training phase.

---

## Feature Engineering & Preprocessing

### Feature Scaling

Because distance metrics like Euclidean distance are heavily influenced by the scale of the data, we must bring features into a similar range.

### Normalization vs. Standardization

| Feature | Min-Max Scaling (Normalization) | Standardization |
|---|---|---|
| **Output Range** | Fixed (0 to 1) | Not bounded |
| **Outlier Handling** | Very sensitive | More robust |
| **Distribution** | Changes the shape | Maintains shape (mostly) |
| **Common Use Case** | When you need exact boundaries from min to max value | When you need a "standard normal curve" (Mean 0, Std Dev 1) |

### Pipelines

- We should avoid altering the entire raw dataset directly before training, as future data would also need to be manually transformed.
- **Solution:** Build the transformation directly into the model using `make_pipeline`.
- Any data that goes into the pipeline automatically passes through the transformation step (e.g., `StandardScaler()`) before reaching the classifier step.

### Feature Engineering & Custom Transforms

The features we provide to a model drastically dictate what it can learn. We can design custom features to give the model better predictive power.

- **Custom Calculations:** Using tools like `FunctionTransformer`, we can add derived features, such as calculating a point's radius/distance from the center $(0, 0)$ to help classify circular data.
- **1D to 2D Signals:** Converting 1-dimensional signals into 2-dimensional representations. For example, converting audio data or stock market data into spectrograms allows models (especially computer vision models) to find complex visual patterns in the data.

---

## The Curse of Dimensionality

### The Problem with High Dimensions

- **Sparsity:** As the number of dimensions (features) increases, data points spread out, increasing sparsity.
- **Failing Distance Metrics:** In high dimensions (e.g., $>10$), Euclidean distance becomes unhelpful because all vectors become almost equidistant from the search query vector. This misleads nearest-neighbor algorithms.

### The Bioinformatics Problem

- In fields like bioinformatics (e.g., genome sequencing), datasets often have very few subjects (e.g., 1000) but millions of features (genes).
- This creates a mathematical problem similar to simultaneous equations: if you have millions of unknown variables (features) but only 1000 equations (subjects), you cannot reliably solve the system.
- **The Solution:** This issue is typically addressed using **Regularization** (especially L1 Regularization, which forces the model to ignore irrelevant features by shrinking their weights to zero).

### Dimensionality Reduction Techniques

To fix the curse of dimensionality before applying algorithms like kNN, we can use:

- **Feature Grouping:** Grouping common features based on domain knowledge (e.g., averaging 365 daily weather readings into 12 monthly averages).
- **Extraction/Embedding Algorithms:** Using techniques like Principal Component Analysis (PCA) or Linear Discriminant Analysis (LDA) to compress the data into a lower-dimensional space.

---

## Decision Trees & Ensembles

### Decision Trees

- **How it works:** Decision Trees mimic human decision-making by incrementally splitting a dataset into smaller subsets based on feature values (e.g., "Is it a mammal? -> Does it have stripes? -> Tiger").
- **Anatomy of a Tree:**
  - **Root Node:** First split, entire dataset.
  - **Decision Node:** Internal split based on a feature condition.
  - **Leaf Node (Terminal Node):** Final node that outputs a prediction.
  - **Depth:** Number of split layers.

### How Trees Learn: Splitting & Purity

- At each node, the algorithm selects the feature and threshold that produce the **purest** child nodes.
- It evaluates all possible splits and chooses the one that minimises impurity.

**Impurity Measures:**

- **Gini Impurity**
- **Entropy (Information Gain)**

**Gini Index (Binary Case):**

$$\text{Gini} = 1 - (p_{yes}^2 + p_{no}^2)$$

- $0 \rightarrow$ perfectly pure node.

- 0.5 → maximum impurity (50/50 split).
- Lower Gini = better split.
- For a full split, compute the **weighted average** of child-node Gini values and choose the lowest.

**Continuous Features:**

1. Sort feature values.
2. Use midpoints as candidate thresholds.
3. Compute impurity for each.
4. Select threshold with minimum impurity.

**Overfitting:**

- Without limits, trees split until leaves are perfectly pure.
- Leads to memorisation and poor generalisation.
- Controlled using hyperparameters such as:
  - `max_depth`
  - `min_samples_split`
  - `min_samples_leaf`

---

**Ensemble Learning**

- **Concept:** Combine multiple models instead of relying on a single tree.
- Leverages the "wisdom of the crowd" for improved accuracy and robustness.

---

**Bagging & Random Forests**

- **Structure:** Models trained in **parallel**.
- **Bagging:**
  - Train models on bootstrap samples (random samples with replacement).
  - Aggregate via majority vote (classification) or averaging (regression).
  - Primarily **reduces variance**.
- **Random Forests:**
  - Bagging + random subset of features at each split.
  - Decorrelates trees so they do not all make the same mistakes.
- **Goal:** Strong variance reduction, robust, hard to overfit.

---

**Boosting**

- **Structure:** Models trained **sequentially**.
- **How it works:**
  - Tree 1 makes predictions.
  - Tree 2 focuses more on misclassified samples.
  - Tree 3 focuses on remaining errors, and so on.
- **Popular Algorithms:** AdaBoost, Gradient Boosting, XGBoost, LightGBM.

- **Goal:** Primarily **reduces bias**.
- Can overfit if not properly tuned.

---

**Bagging vs. Boosting**

- **Bagging:** Parallel, reduces variance, resistant to overfitting.
- **Boosting:** Sequential, reduces bias, more prone to overfitting but often higher performance when tuned well.

### AdaBoost (Adaptive Boosting)

- **Weak Learner:** A model that performs only slightly better than random guessing.
- **Instance Weighting:** Misclassified instances are assigned exponentially higher weights, forcing each subsequent learner to focus on the harder-to-predict samples.
- **Model Weighting:** The final prediction is a weighted majority vote (classification) or weighted sum (regression), where learners with lower training error receive a higher weight.

**Decision Stumps**

- **Definition:** A decision tree with exactly one split ($max_depth = 1$): one root node and two leaf nodes.
- **Role in AdaBoost:** AdaBoost almost exclusively uses decision stumps as its base weak learners because they are fast to train and have high bias / low variance, inherently preventing overfitting at the individual learner level.
- **The Problem with Deep Trees:** Deep trees overfit each individual training iteration, eliminating the error-correction mechanism of boosting and increasing the overall risk of overfitting.

### AdaBoost vs. Random Forest

See Bagging vs. Boosting above for the parallel/sequential and bias/variance distinctions.

- **Base Learner Complexity:**
  - **AdaBoost:** Shallow decision stumps keep variance low; the sequential boosting process progressively reduces bias.
  - **Random Forest:** Unpruned deep trees keep bias low; bagging and random feature selection reduce the resulting variance.
- **Overfitting Risk:** Random Forests are highly robust and difficult to overfit. AdaBoost is sensitive to noisy data and outliers and can overfit if weak learners are too complex or the ensemble is left unchecked.

## Week 5: Gradient Descent

### Training and Loss Functions

Models are trained by finding the parameters (weights) that minimize the error. While we evaluate models using performance metrics like accuracy or $R^2$, we cannot optimize these metrics directly

using calculus. Instead, we define a loss function (or cost function) that measures the difference between model predictions and the correct answers.

- **Mean Squared Error (MSE):**

$$L = \frac{1}{2m} \sum_{i=1}^{m} (y^{(i)} - f_w(x^{(i)}))^2$$

Minimizing this loss function minimizes the overall model error. The objective is to find the global minimum of this function.

## The Gradient Descent Algorithm

Gradient descent is an iterative optimization algorithm used when dealing with tens of thousands of parameters, making exact algebraic solutions computationally inefficient.

### Core Concepts

- **Calculus and Optimization:** Finding the minimum of a function requires taking its partial derivatives with respect to the parameters.
- **The Gradient:** The gradient is a vector formed by taking all the partial derivatives of the loss function with respect to each weight. It points in the direction of the greatest increase of the function.
- **Descent:** Multiplying the gradient by $-1$ provides the direction of the greatest decrease (going "downhill").
- **Weight Update Rule:**

$$w^{t+1} = w^t - \alpha \nabla L(w^t)$$

Where $w^t$ represents the weights at step $t$, $\alpha$ is the learning rate, and $\nabla L$ is the gradient of the loss function.

- **Partial Derivative of MSE Loss (single weight):** To compute the gradient, we differentiate $L$ with respect to each weight using the **chain rule**. For the MSE loss with a single weight $w_1$:

$$\frac{\partial L}{\partial w_1} = \frac{1}{m} \sum_{i=1}^{m} (-x_i)(y_i - w_1 x_i)$$

The $(-x_i)$ factor arises from the chain rule: differentiating the outer squared term gives $2(y_i - w_1 x_i)$ (which cancels with the $\frac{1}{2}$ in the MSE), and then the inner derivative of $(y_i - w_1 x_i)$ with respect to $w_1$ gives $-x_i$. This tells us the direction and magnitude by which $L$ changes as $w_1$ changes. This result is then plugged into the weight update rule above, where the $-\alpha$ term scales it and flips the direction so the weight moves downhill (reducing the loss).

- **Partial Derivative of Log Loss (Logistic Regression):** Gradient descent is not limited to MSE. It applies to any differentiable loss function. For logistic regression, we minimise the log loss (see Week 3) instead. Taking the partial derivative of log loss with respect to weight $w_j$ yields:

$$\frac{\partial L}{\partial w_j} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)\, x_j^{(i)}$$

where $\hat{y}_i = \sigma(w \cdot x^{(i)})$ is the sigmoid output. Despite coming from a completely different loss function, the gradient has the same intuitive structure as the MSE case (error times input feature), and is plugged into the same weight update rule to train the logistic regression model.

- **Multivariate Weight Update:** Generalising to multiple weights, each weight $w_j$ is updated as:

$$w_j \leftarrow w_j - \alpha\frac{1}{m}\sum_{i=1}^{m} x_j^{(i)}(f_w(x^{(i)}) - y^{(i)})$$

**Algorithm Steps**

1. Initialize the weights to random values or zero.
2. Calculate the loss function.
3. Calculate the gradient (partial derivative of the loss with respect to each weight).
4. Update all weights simultaneously using the weight update rule.
5. Repeat the procedure iteratively until convergence (when the result stops changing).

**Potential Issues**

- **Local Minima:** In complex, multi-modal loss landscapes, the algorithm might converge to a local minimum instead of the optimal global minimum.

## Gradient Descent Variations

Gradient descent varies based on how many training patterns (samples) are used to calculate the error before updating the model weights. This creates a trade-off between computational efficiency and gradient stability.

- **Batch Gradient Descent (Full Batch):** Uses every sample in the training set to calculate a single loss and performs one update. The batch size equals the entire training set size, so **one epoch equals exactly one weight update**.
- **Stochastic Gradient Descent (SGD):** Updates the model after each individual sample (batch size = 1). With $m$ samples, this means $m$ **weight updates per epoch**. Introduces randomness and high variance into the loss trajectory.
- **Mini-Batch Gradient Descent:** Divides the dataset into small batches and updates weights after each batch. This is the standard practical approach as it balances stability and speed. The number of updates per epoch is:

$$\text{Updates per Epoch} = \frac{\text{Total Sample Size}}{\text{Batch Size}}$$

**What does "Stochastic" mean?** Stochastic means random. Both SGD and Mini-Batch GD are stochastic processes because they randomly sample data before computing the gradient. As long as you are randomly sampling (whether it is one row or a batch of 32), the gradient computed is an **estimate** of the true gradient rather than the exact value. Only Full Batch GD uses the entire dataset, making its gradient exact. This is the fundamental distinction: Full Batch computes the truth; SGD and Mini-Batch estimate it.

**Note on Naming:** Despite true SGD having a batch size of 1 (above), most modern ML frameworks (e.g., PyTorch's `torch.optim.SGD`, Keras) implement mini-batch gradient descent under the label "SGD". In practice, when someone says they are "using SGD", they almost always mean mini-batch.

## Epochs, Batches, and Learning Rates

### Hyperparameters

- **Batch Size:** The number of samples processed before the model updates its weights.
- **Epoch:** One complete pass through the entire training dataset, ensuring every individual sample has been seen by the model at least once. One epoch is comprised of one or more batches. Algorithms typically run for multiple epochs.
- **Learning Rate ($\alpha$):** Determines the step size at each iteration while moving toward a minimum.

### Optimizing the Learning Rate

Choosing the correct learning rate is critical for model convergence.

- **Too High:** The algorithm steps too far, potentially overshooting the minimum and causing the loss to diverge.
- **Too Low:** The algorithm takes tiny steps, requiring excessive epochs and computational time to converge.

### Optimization Methods

Various advanced optimization techniques build upon standard SGD to adjust learning rates and momentum for better performance and to avoid getting stuck in local minima:

- SGD with Momentum
- Nesterov Momentum
- AdaGrad
- RMSProp
- Adam (The default solver in many libraries, such as Scikit-Learn)

# Week 6: Regularisation and Support Vector Classifiers

## Regularisation

### Concept and Motivation

- Definition: A technique used in machine learning models to reduce overfitting, improve model generalization, and manage model complexity by adding an additional penalty term to the error function.
- Regularisation restricts fluctuating parameters so that coefficients do not take extreme values.
- This technique is also referred to as a shrinkage method or weight decay (particularly in neural networks).
- Overfitting implies the model has high variance, meaning the model will change significantly if the training data is altered.
- Increased parameters and flexibility cause high variance.
- Extremely large weights can change by massive amounts very quickly, making the model overly sensitive to small changes in features.
- Regularisation solves this by adding a penalty to the loss function that scales with the size of the weights.

### Mean Squared Error

- The Mean Squared Error (MSE) is commonly used as a loss function for regression models.
- **Mean Squared Error Formula:**

$$MSE(w) = \frac{1}{2m} \sum_{i=1}^{m} (y_i - f_w(x_i))^2$$

- Large weights may emerge when minimizing MSE, which can increase the importance of particular features and lead to overfitting.

### L2 Regularisation (Ridge Regression)

- L2 regularisation adds a penalty term based on the L2 norm (Euclidean distance) to the loss function to penalize large weights.
- **L2 Regularised Loss Function:**

$$L(w) = MSE(w) + \lambda \sum_{j=1}^{n} w_j^2$$

- The bias term ($w_0$) is conventionally excluded from the regularisation penalty.
- A trained L2 regularised model will be a slightly worse fit for the training data but will successfully avoid overtraining by adding bias and reducing variance.
- The hyperparameter $\lambda$ controls the trade-off between the original loss and the penalty:
  - A small $\lambda$ prioritizes minimizing the original cost function, moving the model in the direction of overfitting.
  - A large $\lambda$ prioritizes small weights, moving the model in the direction of underfitting.

- Gradient descent still applies with regularisation, but the gradient includes an additional term caused by the regularisation penalty.

### L1 Regularisation (Lasso Regression)

- L1 regularisation adds a penalty term based on the L1 norm (Manhattan distance), using the absolute value of the coefficients rather than squaring them.
- **L1 Regularised Loss Function:**

$$L(w) = MSE(w) + \lambda \sum_{j=1}^{n} |w_j|$$

- L1 norm corresponds to Manhattan distance, while L2 norm corresponds to Euclidean distance.
- The key operational difference between L1 and L2 is that Lasso Regression can shrink weights of non-relevant features exactly to zero, effectively excluding them from the model.
- This increases sparsity, making it highly effective for datasets with many irrelevant features (e.g., biomedical datasets with irrelevant biomarkers).
- Ridge Regression generally performs better when the majority of variables are known to be useful.

### Elastic Net Regularisation

- Elastic Net is a third type of regularisation that combines both L1 (Lasso) and L2 (Ridge) penalties into a single loss function.
- This allows the model to benefit from both feature selection (L1) and coefficient shrinkage (L2) simultaneously.

### Advantages of Regularisation

- Regularisation generally improves model performance under the following conditions:
  - Multicollinearity: When features are highly correlated, regularisation prevents overfitting.
  - High Dimensionality: When there are a large number of features (especially when features outnumber observations), it shrinks coefficients and selects relevant features, keeping variance low despite limited data.
  - Irrelevant Features: It forces the coefficients of non-contributing features toward zero.
  - Noisy Data: It reduces the model's sensitivity to noise by preventing the model from over-adjusting to noisy feature variations.

## Support Vector Classifiers

### Generalised Linear Models and Geometry

- The standard equation of a line ($y = mx+c$) can be generalized into a vector equation format.
- **Linear Boundary Vector Equation:**

$$w^T x + w_0 = 0$$

- Evaluating a sample point $x$ using $w^T x + w_0$ yields a scalar value. The sign of this scalar (positive or negative) determines which side of the line the point falls on, which is the basis for classification.

**Hyperplanes and Decision Boundaries**

- A linear classifier utilizes a linear boundary, known as a hyperplane, which separates the feature space into two half-spaces.
- The geometric interpretation of a hyperplane depends on the dimensionality of the feature space:
  - 1D space: A threshold point.
  - 2D space: A line.
  - 3D space: A flat plane.
- The vector $w$ represents a normal vector that points orthogonally (perpendicularly) away from the hyperplane surface.
- **Dot Product Orthogonality Condition:**

$$a \cdot b = |a||b| \cos \theta$$

- The dot product of two vectors is zero only if they are entirely orthogonal to one another.
- The equation $w^T x = 0$ represents a hyperplane passing through the origin, while adding the bias term $w_0$ shifts the hyperplane away from the origin.

**Separability and Margins**

- A classification problem is linearly separable if a hyperplane can be drawn that perfectly separates the classes.
- Non-perfect separation in training data can be caused by:
  - The chosen model being too simple to capture the relationship.
  - Noise or mislabeled targets in the dataset.
  - Simple features that fail to account for all data variations.
- A robust decision boundary is selected by maximizing the margin between the boundary and the different classes.
- Definition: The margin is defined as the shortest geometric distance between the closest observations and the separating hyperplane.
- A Maximal Margin Classifier (or Hard Margin Classifier) enforces a strict boundary that strictly maximizes the distance between the classes without allowing any misclassifications.
- Hard Margin Classifiers are highly sensitive to outliers and prone to severe overfitting, requiring relaxed constraints for functional real-world application.

# Week 7: Support Vector Classifiers and Machines

## Margins and Decision Boundaries

- **Margin:** The shortest geometric distance between the closest observations and the separating hyperplane.

- **Support Vectors:** The specific training samples that lie closest to the decision boundary. They are the only points that influence the position of the hyperplane.
- The goal of a Support Vector Classifier (SVC) is to find a decision boundary that maximises this distance to both classes.
- **Distance from a Point to the Decision Boundary:**

$$D(x) = \frac{w^T x + w_0}{||w||}$$

- **Margin Equation:**

$$m = \frac{2}{||w||}$$

- **Hyperplane Scaling Insight:** We can rescale the hyperplane so that support vectors lie on:

$$w^T x + w_0 = \pm 1$$

- **Hard Margin Optimisation Problem:**

$$\min_w ||w||^2 \text{ such that } y^{(i)}(w^T x^{(i)} + w_0) \geq 1$$

- A Hard Margin Classifier strictly maximises the margin while maintaining zero misclassifications, making it highly sensitive to outliers and prone to overfitting.

## Soft Margin Classifiers and Slack Variables

- To prevent overfitting and handle noisy data, the strict constraints of a hard margin can be relaxed using a Soft Margin Classifier.
- There is a fundamental **trade-off between maximising the margin and minimising classification error**.
- **Soft Margin Optimisation Problem:**

$$\min_w ||w||^2 + C \sum_{i=1}^{N} \xi_i$$

- **Constraints:**

$$y^{(i)}(w^T x^{(i)} + w_0) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- **Slack Variables ($\xi_i$):** Added to each sample to allow for margin violations.
  - If $\xi = 0$: The point is on the correct side of the margin
  - If $0 < \xi \leq 1$: The point violates the margin but is still correctly classified
  - If $\xi \geq 1$: The point is misclassified

- **Regularisation Parameter ($C$):** Controls the trade-off between maximising the margin and minimising classification errors.
  - Small $C$: Constraints are easily ignored, resulting in a wider margin
  - Large $C$: Constraints are strictly enforced, resulting in a narrow margin
  - $C \to \infty$: Enforces all constraints, reverting to a Hard Margin Classifier
  - Larger $C$ can also increase computational cost during training

### Hinge Loss

- Support Vector Machines utilize the Hinge Loss function to evaluate classification error.
- **Hinge Loss Function:**

$$L(y^{(i)}, f(x^{(i)})) = \max(0, 1 - y^{(i)}(w^T x^{(i)} + b))$$

- **Connection to Slack Variables:**

$$\xi_i \geq 1 - y^{(i)}(w^T x^{(i)} + b)$$

### Support Vector Machines and Higher Dimensions

- Some classification problems are not linearly separable in their original feature space.
- **Mapping:** By projecting the data into a higher-dimensional space using a transformation function $\Phi(x)$, a linear hyperplane can often be found to separate the classes.
- Example: XOR problem (not linearly separable in 2D, but separable in higher dimensions)
- However, explicitly converting points to a high-dimensional space significantly increases computational complexity and time.

### The Kernel Trick

- The Kernel Trick allows the algorithm to learn a classifier in a high-dimensional feature space without ever explicitly mapping the points into that space.
- It relies on converting the optimisation problem from its Primal form to its Dual form.
- **Primal Form Classifier:**

$$f(x) = w^T x + b$$

- **Dual Form Classifier:**

$$f(x) = \sum_{i=1}^{N} \alpha_i y_i (x_i^T x) + b$$

- **Dual Constraints:**

$$0 \leq \alpha_i \leq C, \quad \sum_{i=1}^{N} \alpha_i y_i = 0$$

- In the Dual form, the calculation only depends on the dot product of the input vectors ($x_i^T x_j$).
- Only points with $\alpha_i \neq 0$ contribute to the classifier — these are the **support vectors**.
- The Kernel Function $k(x_i, x_j)$ directly computes this dot product in the higher-dimensional space without performing the expensive transformation.
- **Kernel Example:**

$$k(x_i, x_j) = (x_i^T x_j)^2$$

- **Complexity:** Learning complexity relies on $N$ (the number of training samples) rather than $D$ (the dimensionality), making the process highly efficient.

### Types of Kernels

- **Linear Kernel:** Standard dot product, used when data is linearly separable.
  - $k(x_1, x_2) = x_1^T x_2$
- **Polynomial Kernel:** Contains all polynomial terms up to a specified degree $d$.
  - $k(x_1, x_2) = (x_1^T x_2 + c)^d$
  - The hyperparameter $c$ controls the influence of non-linear terms versus linear terms
  - Larger $c$ increases the influence of lower-order (smoother) terms
- **Radial Basis Function (RBF) / Gaussian Kernel:** Maps data into an infinite-dimensional feature space.
  - $k(x_1, x_2) = \exp(-\gamma ||x_1 - x_2||^2)$
  - The hyperparameter $\gamma$ (derived from variance $\sigma$) dictates how closely the model behaves like a nearest-neighbour classifier
  - Large $\gamma \rightarrow$ very local decision boundary (similar to nearest neighbour)
- **Sigmoid Kernel:**
  - $k(x_1, x_2) = \tanh(x_1^T x_2 + \theta)$

### Implementation Tips

- **Scaling:** SVMs are not scale-invariant. You must normalize or scale your data (e.g., using `StandardScaler` or `MinMaxScaler`) to ensure features have equal weighting.
- **Parameter Tuning:** Identifying the optimal kernel type, $\gamma$, and $C$ parameter is crucial. Tools like `GridSearchCV` are typically used to test combinations efficiently via Cross-Validation.

## Week 8: Neural Networks and Deep Learning Frameworks

### Artificial Neural Networks

- Artificial Neural Networks (ANNs) are machine learning models inspired by the biological structure of human and animal brains.

- The human brain consists of approximately $10^{11}$ neurons, each communicating with thousands of other neurons. ANNs attempt to replicate this highly interconnected computational power mathematically.
- While inspired by biology, modern neural network design prioritizes mathematical efficiency and predictive performance over strict biological plausibility.
- Neural networks excel at solving complex, non-linear problems that traditional algorithms struggle with, such as image recognition, natural language processing, and advanced pattern recognition.
- The foundational architecture for many of these tasks is the Multi-Layer Perceptron (MLP).

## The Single Perceptron

- The perceptron is the fundamental computational unit (neuron) of an artificial neural network.
- It receives one or more input values, computes a weighted sum of these inputs, adds a bias term, and passes the result through an activation function to produce an output.
- **Perceptron Linear Combination:**

$$z = \sum_{i=1}^{n} w_i x_i + b$$

- $x_i$ represents the input features.
- $w_i$ represents the learned weights, determining the importance of each input.
- $b$ represents the bias term, which shifts the activation function allowing the model to fit data that does not pass through the origin.
- $z$ is the resulting linear output, which is then fed into an activation function.

## Activation Functions

- Activation functions introduce non-linear properties to the network. Without them, a neural network, regardless of its depth, would simply behave as a linear regression model.

### Threshold Function (Step Function)

- A simple binary function that outputs 1 if the input exceeds a certain threshold, and 0 otherwise.
- It is rarely used in modern deep learning because it is not differentiable, preventing the use of gradient descent for training.

### Sigmoid Function

- Squeezes input values into a continuous range between 0 and 1.
- Historically popular, especially for binary classification output layers (representing probability).
- Suffers from the vanishing gradient problem, where large positive or negative inputs cause the gradient to approach zero, stalling the learning process in deep networks.

### Hyperbolic Tangent (tanh)

- Similar to the sigmoid function but squeezes inputs to a range between $-1$ and 1.
- Because it is zero-centered, it generally performs better than the sigmoid function for hidden layers, though it still suffers from vanishing gradients at extreme values.

### Rectified Linear Unit (ReLU)

- Currently the most widely used activation function for hidden layers in deep learning.
- It outputs the input directly if it is positive; otherwise, it outputs zero.
- **ReLU Function:**

$$f(z) = \max(0, z)$$

- Advantages: Computationally highly efficient and significantly accelerates the convergence of gradient descent compared to sigmoid or tanh.
- Disadvantages: Can suffer from the "dying ReLU" problem, where neurons get pushed into states where they output zero for all inputs and stop updating completely. Variations like "Leaky ReLU" are used to mitigate this.

## Network Topology

- Neural networks are organized into distinct layers of interconnected nodes.
- **Input Layer:** Receives the raw initial data. The number of nodes corresponds to the number of features in the dataset.
- **Hidden Layers:** Intermediate layers where the core computations and feature extractions occur. A network with multiple hidden layers is referred to as a "Deep" Neural Network.
- **Output Layer:** Produces the final prediction. The structure depends on the task (e.g., one node with a sigmoid activation for binary classification, multiple nodes with softmax for multi-class classification).
- **Feedforward Mechanism:** Data flows strictly in one direction, from the input layer, through the hidden layers, to the output layer, without looping back.

## Deep Learning Frameworks: TensorFlow and Keras

### TensorFlow

- An open-source symbolic mathematics library developed by Google Brain.
- It is the dominant, industry-standard framework for building and training neural networks.
- It is highly optimized for executing fast, large-scale linear algebra operations (especially matrix multiplications) using CPUs, GPUs, or TPUs.

### Keras

- A high-level, open-source neural network API written in Python.
- Designed for human usability, providing a consistent and simple interface that abstracts away the complex, low-level mathematics of the backend engine.
- Keras typically runs on top of TensorFlow as its default backend computation engine.

### Building and Training Models in Keras

#### Compiling the Model

- Before a model can be trained, it must be compiled with specific configurations.
- **Optimizer:** The algorithm used to update the network weights (e.g., Adam, Stochastic Gradient Descent).
- **Loss Function:** The mathematical metric the optimizer attempts to minimize (e.g., `binary_crossentropy` for binary classification, `mean_squared_error` for regression).
- **Metrics:** Human-readable metrics used to evaluate the model's performance during training and testing (e.g., `accuracy`).

#### Fitting the Model (Training)

- The `model.fit()` method initiates the training process.
- **Epochs:** A hyperparameter defining the number of complete passes the learning algorithm makes through the entire training dataset.
- **Batch Size:** The number of training samples processed simultaneously before the model's internal parameters (weights) are updated.
- **Validation Split:** A parameter that reserves a specified fraction of the training data to evaluate the loss and metrics at the end of each epoch.
  - *Warning:* Keras takes the validation split directly from the end of the provided dataset without shuffling. If the data is sequentially ordered, this will result in poor validation metrics. Data must be shuffled prior to this step.

#### Evaluating and Predicting

- **Evaluation:** The `model.evaluate()` method tests the fully trained model against an unseen testing dataset to calculate the final loss and metrics. This operation is computationally faster than fitting.
- **Prediction:** The `model.predict()` method takes a new vector of features ($X_{new}$) and outputs the network's calculated predictions (e.g., class probabilities or continuous regression values).

## Week 9: Convolutional Neural Networks

### Background and Motivation

- **The Curse of Dimensionality:** Images contain a massive amount of data, making them extremely high-dimensional problems. For example, a tiny $32 \times 32 \times 3$ color image represents a 3072-dimensional problem.
- **Spatial Invariance:** In images, features are not independent of their position; the 2D structure matters. If an object is in the image, it should be recognized regardless of its location. Convolutional Neural Networks (CNNs) leverage this by using spatial invariance.
- **Intuition of Representation Learning (The Clock Example):**
  - If we want a model to read the time from an image of a clock, using raw pixels requires an extremely complex model.
  - A better representation (using edge detection) extracts the coordinates of the clock hands.

  - An even *better* representation extracts the mathematical angles ($\theta$) of the hands.
  - Deep learning aims to automatically discover these intermediate representations layer-by-layer, rather than making the jump from raw pixels to the final answer in one step.
- **Representation Learning:** Unlike conventional approaches that use hand-crafted algorithms to detect edges (Sobel, Canny) or corners (Harris, SIFT), Deep Learning allows the algorithm to learn its own intermediate representations.
- **Transfer Learning:** Lower-level features learned by CNNs are often universally useful for similar tasks, allowing models to be fine-tuned for new tasks without training from scratch.

## CNN Concepts and Layers

A ConvNet transforms an input 3D volume to an output 3D volume using differentiable functions across multiple layers. These networks closely resemble the early parts of the human visual cortex.

### Convolutional Layer

- **Filters and Sliding:** The layer uses small filters (kernels) that slide across the width and height of the image, computing the dot product at each position.
- **Weight Sharing:** The weights of the filter are shared across all positions, meaning the filter learns the best response taking every position into account.
- **Multiple Filters:** A single filter looks for one specific feature. CNNs use multiple filters (e.g., 10 filters) to produce multiple activation maps, transforming the depth of the representation.
- **Voxels:** While standard 2D images use pixels, the 3D grid structures (width, height, depth) processed within CNN volumes can be conceptualized as being made up of voxels (3D pixels).
- **Zero Padding:** Zeros are added around the border of the input to control the spatial size of the output volumes and ensure the filter can process edge pixels properly.
- **Stride:** The number of steps the convolutional filter moves across the input feature map. A larger stride results in less overlap and a smaller output volume.

**Output Width Calculation:**

$$W_l = \frac{W_{l-1} - F_l + 2P_{l-1}}{S_l} + 1$$

**Output Height Calculation:**

$$H_l = \frac{H_{l-1} - F_l + 2P_{l-1}}{S_l} + 1$$

**Output Depth Calculation:**

$$D_l = K_l$$

Where $W$ is width, $H$ is height, $D$ is depth, $F$ is filter size, $P$ is padding, $S$ is stride, $K$ is the number of filters, and $l$ is the layer number.

- **Popular Filter Sizes & 1x1 Convolutions:**
  - The most common filter sizes are $3 \times 3$ and $5 \times 5$. Filters of $7 \times 7$ or larger are rarely used.

- $1 \times 1$ **Filters:** While a $1 \times 1$ filter has no spatial spread, it is highly useful. Because convolution applies to the full depth of the previous layer, a $1 \times 1$ filter acts as a weighted average of the activation maps from the previous layer. It is essentially a dimensionality reduction tool along the depth axis.

#### Pooling Layer

- **Dimensionality Reduction:** CNNs progressively reduce the spatial dimensions of the representation to reduce the number of parameters and computational complexity.
- **Max-Pooling:** Takes the maximum value from a defined region (e.g., $2 \times 2$ grid with a stride of 2). This is most popular in recognition and classification networks.
- **Average-Pooling:** Takes the average value from a defined region. More common in generative networks.

#### Fully Connected Layer

- **Purpose:** After extracting features and reducing dimensionality through convolutional and pooling layers, the data is flattened and passed into Fully Connected (Dense) layers.
- **Classification:** These layers map the learned representation to the final output, typically feeding into a multi-class classifier (like softmax) or an SVM.

### CNN Training and Overfitting Prevention

Due to the massive number of parameters, CNNs are prone to overfitting. Standard techniques like regularisation and early stopping are used, along with specific deep learning strategies.

#### Data Augmentation

- **Definition:** Expanding the training dataset by creating modified copies of the original images.
- **Techniques:** Random cropping, scaling, rotations, translations, adding noise, and lens distortions.
- **Benefit:** Exposes the model to more variations, making it robust and less likely to memorize the exact training images.

#### Dropout

- **Definition:** Randomly dropping out (setting to zero) a percentage of output units (e.g., 20% or 40%) in a layer during the training process.
- **Benefit:** Prevents the network from becoming overly reliant on specific neurons or filters, forcing it to learn redundant and robust representations.
- **Inference:** Dropout is disabled during testing/prediction; all units are active.

### Famous Architectures

#### LeNet

- **Year:** 1989 (Published 1998).
- **Significance:** First successful use of a CNN, created by Yann LeCun. Used extensively by postal services to read handwritten zip codes.

**AlexNet**

- **Year:** 2012.
- **Significance:** The major breakthrough for CNNs, winning the ImageNet challenge by a huge margin. It proved the viability of deep CNNs.
- **Structure:** Comprised 8 layers ($227 \times 227 \times 3$ input) using ReLU activations, heavy data augmentation, and Dropout. It had to be split across two GPUs due to hardware constraints.

**VGGNet**

- **Year:** 2014.
- **Significance:** Standardized the architecture by using uniform $3 \times 3$ convolutions (stride 1, pad 1) and $2 \times 2$ max-pooling.
- **Depth:** Went deeper, ranging from 11 to 19 layers (VGG-16 is highly popular). The fully connected layer weights are widely used today as a baseline feature extractor for transfer learning.

**GoogLeNet (Inception)**

- **Year:** 2014.
- **Significance:** Won the 2014 ImageNet competition by drastically reducing the number of parameters from 60 million (AlexNet) to just 4 million, improving computational efficiency.

**ResNet**

- **Year:** 2015.
- **Significance:** Introduced Residual Networks by Microsoft to solve the vanishing gradient problem associated with extreme depth.
- **Depth:** Successfully trained networks with 50, 101, and 152 layers. It was the first model to beat human-level performance on the ImageNet dataset.
- **The Degradation Problem (Why ResNet was needed):**
  - Empirical evidence showed that simply stacking more layers does *not* necessarily mean better accuracy.
  - A famous experiment compared a 20-layer network to a 56-layer network. The 56-layer network had significantly higher training and test errors.
  - ResNet solved this "vanishing gradient" / degradation problem by introducing **Residual Learning** (skip connections), allowing networks to successfully train at extreme depths (up to 152 layers).

# Week 10: Neural Networks with Multiple Outputs & Transfer Learning

## Neural Networks with Multiple Outputs

### Terminology: Logits

- In the context of Logistic Regression, the logit function converts a probability ($p$) into the log of the odds.

- In the context of Deep Learning, a logit refers to the raw, unnormalized prediction values that come directly out of the output layer of a neural network before any activation function is applied.
- These raw logits potentially range from $-\infty$ to $+\infty$.

**The Softmax Function**

- When dealing with multi-class problems (e.g., classifying an image as an apple, banana, or car), a binary activation function like Sigmoid is insufficient.
- The Softmax function is used in the final layer of a neural network classifier to generalize the Sigmoid function to multiple classes.
- It converts a vector of raw logits into a vector of probabilities that sum exactly to 1.0.
- **Softmax Function:**

$$S(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

- The output of the Softmax function allows the model to express a normalized probability distribution across all possible, mutually exclusive categories.

**Multi-Label Classification**

- Multi-class classification assumes each sample belongs to exactly one category. Multi-label classification occurs when a single sample can belong to multiple categories simultaneously (e.g., an image containing an apple, a dog, and candy).
- Because the categories are not mutually exclusive, the probabilities across all classes do not need to sum to 1.0.
- Therefore, Softmax cannot be used. Instead, multiple Sigmoid activation functions are used at the output layer (one for each independent binary classification task).

**Error Metrics and Cross Entropy Loss**

- Mean Squared Error (MSE) is primarily used for regression tasks and performs poorly for classification.
- Log Loss is used for binary classification.
- For multi-class neural networks, Log Loss is extended into Cross Entropy Loss.
- Cross Entropy Loss calculates the error based on the predicted probability of the *correct* class.
- **Simplified Cross Entropy Loss:**

$$L = -\log(P_{\text{correct}})$$

- Where $P_{\text{correct}}$ is the network's predicted probability for the actual true label. If the model predicts a high probability for the correct class, the loss approaches zero. If the model predicts a low probability for the correct class, the loss penalizes it heavily.

## Transfer Learning

### Introduction to Transfer Learning

- Transfer learning is a machine learning technique where a model originally trained on one task is reused as the starting point for a model on a different, but related, task.
- Instead of training a deep neural network from scratch (random weight initialization), it leverages the feature representations already learned by an established model on a massive dataset.

### Benefits of Transfer Learning

- **Data Efficiency:** Enables effective learning and high accuracy even with significantly smaller target datasets.
- **Faster Training:** Starting with pre-trained weights drastically reduces the computational time required to achieve convergence.
- **Improved Performance:** Generally yields better overall predictive performance and robustness, especially in domains where labeled data is scarce.

### The Transfer Learning Process

The standard implementation process generally follows these steps:

1. **Obtain a Pre-Trained Model:** Select an established architecture (e.g., VGG16, ResNet) that has been trained on a large-scale dataset (e.g., ImageNet).
2. **Create a Base Model:** Load the network without its original final classification layers (the "top" or "head" of the network).
3. **Freeze Base Layers:** Lock the weights of the pre-trained base model so they do not update during the initial training phase.
4. **Add Custom Layers:** Append new, randomly initialized layers suited to the specific target task (e.g., a Global Average Pooling layer followed by a Dense layer with a Softmax activation).
5. **Train the New Layers:** Train the model on the new dataset. Only the weights in the newly added custom layers will be updated.

### Fine-Tuning

- After the initial training of the custom layers, the model can be further optimized through fine-tuning.
- **Unfreezing:** The base model (or the top few layers of the base model) is unfrozen, allowing its weights to be updated.
- **Low Learning Rate:** The model must be recompiled and trained using a severely reduced learning rate (e.g., `1e-5`). Using a standard learning rate would cause massive gradient updates that destroy the delicate, pre-trained representations.
- **Inference Mode:** When building the model, it is crucial to pass `training=False` to the base model. This ensures that layers like Batch Normalization run in inference mode and do not improperly update their internal mean and variance statistics during the fine-tuning stage.

# Week 11: Backpropagation and Neural Network Training Parameters

## Backpropagation

### Historical Context and Usage

- **Origins:** Backpropagation for training multiple layers was discovered by Seppo Linnainmaa in 1970 and later popularized by researchers including Rumelhart and Hinton in the 1980s, which kickstarted modern deep learning research.
- **Biological Plausibility:** Unlike the structural inspiration of artificial neural networks, backpropagation is not biologically plausible. The physical hardware of the human brain is not suited to performing this specific mathematical algorithm.
- **Modern Applications:** It is the foundational training mechanism used to optimize networks across diverse domains, including Natural Language Processing (Deep LLMs like ChatGPT), speech processing, object detection (e.g., YOLO), and fine-tuning pre-trained models for transfer learning.

### The Algorithm

- **Forward Pass:** The network computes a prediction by passing input data through sequential layers of weights and activation functions.
  - Linear combination function: $f = W \cdot X$
  - Two-layer network (with ReLU): $f = W_2 \cdot \max(0, W_1 \cdot X)$
  - Three-layer network: $f = W_3 \cdot \max(W_2 \cdot \max(0, W_1 \cdot X))$
- **Loss Computation:** The final predicted output is compared against the true target label using a loss function to determine the total error.
- **Backward Pass:** The algorithm calculates the gradient of the loss function with respect to every single intermediate variable, weight, and bias by working backwards from the output layer to the input layer.
- **The Chain Rule:** This is the core calculus principle that enables backpropagation. It allows for the calculation of derivatives for nested activation functions by iteratively multiplying local gradients.
- **Chain Rule:**

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

## Neural Network Training Parameters

### Network Architecture Parameters

- **Depth of Network:** The total number of hidden layers. Larger, deeper networks are better able to model highly complex, non-linear functions but require stricter regularisation to prevent severe overfitting.
- **Number of Units:** The width of each specific layer, dictating the dimensionality of the learned intermediate representations.

- **Activation Function:** The non-linear transformation applied to the outputs of each layer (e.g., ReLU, Sigmoid, Tanh).
- **Weight Initialisation:** The statistical strategy used to set the starting values of the network's weights before optimization begins.

**Optimization and Learning Constraints**

- **Learning Algorithm (Optimizer):** The mathematical solver used to update the weights based on the computed gradients.
  - **SGD (Stochastic Gradient Descent):** The classic optimization approach.
  - **Adam:** An optimized, adaptive learning rate version of SGD. It is the default solver in most modern frameworks (like scikit-learn's `MLPClassifier`).
  - **L-BFGS:** A quasi-Newton optimization method that often yields better performance on smaller datasets compared to SGD.
- **Loss Function:** The mathematical objective being minimized (e.g., Log-Loss/Cross-Entropy for classification tasks).
- **Learning Rate:** A scalar hyperparameter that controls the step size of the weight updates during gradient descent.
- **Batch Size:** The number of individual training samples processed simultaneously before the model performs a weight update.
- **Number of Epochs:** The total number of complete passes the training loop makes over the entire dataset.
- **Regularisation Parameter:** The penalty applied to large weights to enforce simplicity, reduce model variance, and mitigate overfitting.
- **Early Stopping:** A defensive training mechanism that automatically halts the optimization process when validation set performance stops improving.

**Framework and Vectorization**

- **Fully-Connected Layers:** Neurons are arranged in dense, fully-connected structures to allow for highly efficient code execution.
- **Vectorized Code:** By abstracting neurons into distinct layers, operations are executed simultaneously as large-scale matrix multiplications rather than individual calculations.

**Model Selection Considerations**

- **Data Dependency:** Deep learning networks represent the state-of-the-art but require massive amounts of labelled data. If a network requires 10000 points to properly train but only 1000 are available, the model is practically useless.
- **Alternative Approaches:** Data augmentation and synthesis can be used to artificially inflate dataset size in low-data scenarios.
- **Traditional Machine Learning:** If data is strictly limited, classic algorithms (Naive Bayes, kNN, SVM, Random Forests) often outperform deep neural networks and remain a vital part of the machine learning toolkit.